

Prise en main du robot ROSKit

Documentation Technique

4 mai 2026

Table des matières

1	Interagir avec le robot	2
1.1	Composition du robot	2
1.2	Comment allumer le robot	2
1.3	Télécommande	3
1.4	Connexion Wifi et SSH	4
2	Navigation Autonome avec Nav2	8
2.1	Packages nécessairent	8
2.2	Conversion PointCloud vers LaserScan	8
2.3	Cartographie (SLAM)	9
2.4	Lancement de la navigation	10
2.5	Enregistrement de la carte	10

1 Interagir avec le robot

1.1 Composition du robot

Le robot est équipé d'un :

- Scout Mini / Bunker Mini
- Asus NUC 15 Pro CRK
- Router Teltonika RUT956
- GPS : simpleRTK2B Ardu Simple
- IMU : Phidgetspatial
- RealSense D435
- LiDAR : Robosense Helios 16P

1.2 Comment allumer le robot

Appuyez sur le bouton de démarrage à l'arrière du robot. Le bouton s'allumera en rouge et l'écran affichera la batterie et le voltage.



FIGURE 1 – Arrière du Robot

1.3 Télécommande

Pour télécommander le robot, prenez la manette. Vous disposez de deux joysticks et 4 interrupteurs. Les interrupteurs n°2 et 3 ont trois crans, les autres en ont que 2. En plaçant tous les **interrupteurs en position haute**, maintenir les deux boutons power pendant quelques secondes. Des sons devraient retentir, la télécommande est allumée.



FIGURE 2 – Télécommande

Déplacement : Abaissez d'un cran le interrupteur n°2 et n°3. Le n°2 permet de retirer la sécurité, le n°3 la vitesse. Additionnelement, le n°4 permet éteindre ou d'allumer les feux avant.

Vitesse (3ème interrupteur) :

- Position milieu : Vitesse lente
- Position haute : Vitesse medium
- Position basse : Vitesse haute

1.4 Connexion Wifi et SSH

Pour travailler sur le robot, il vous faut attendre quelques secondes que le router s'allume. Chercher le réseau du robot en wifi. Le nom du robot est inscrit sur le robot, il devrait s'appeler **roskit_ "numéro du robot"**.

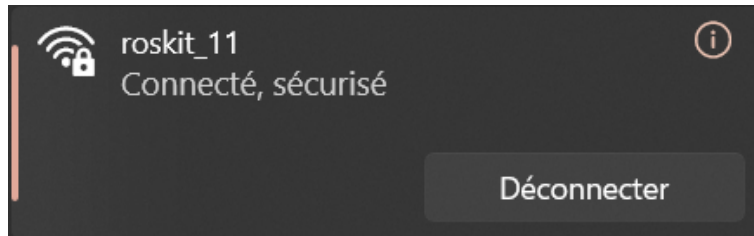


FIGURE 3 – Connexion Wifi au robot

Si ce nom n'apparaît pas c'est que **le robot a été réinitialisé**, il faut aller sur le manuel d'utilisation et chercher le nom correspondant au robot. Une fois connecté au robot, nous allons nous connecter à internet.

Connection internet : Pour se connecter à internet, on va se rendre sur <https://192.168.1.1>. Les identifiants sont dans le **manuel** section 6.1 : Route Page.

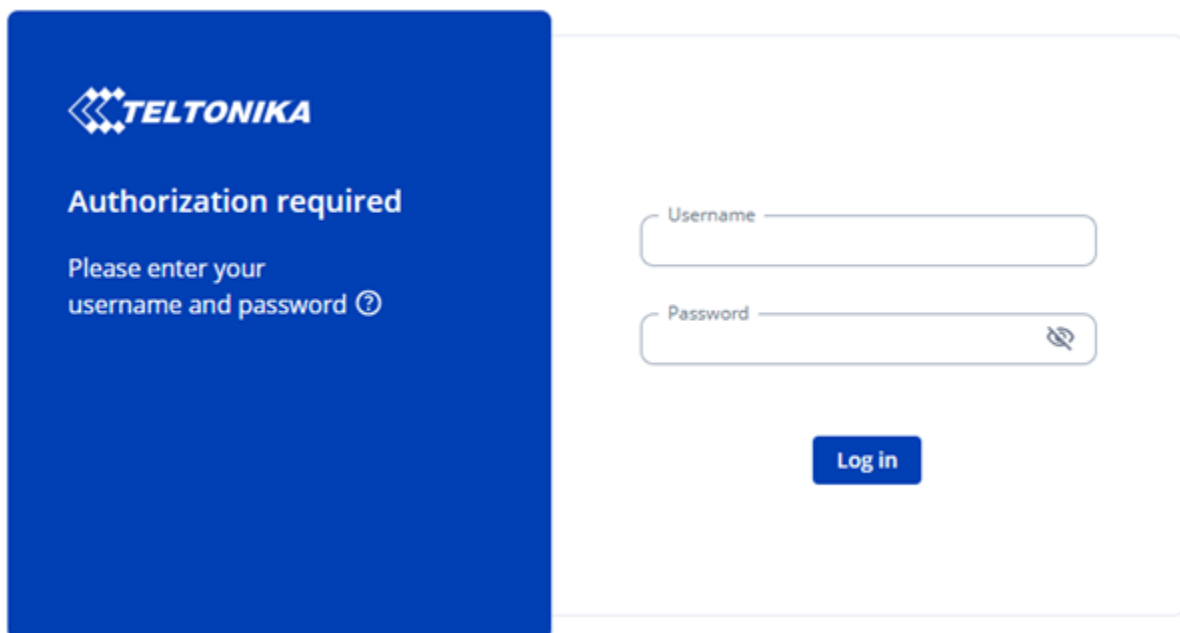


FIGURE 4 – Page d'accueil de teltonika

Une fois sur le site, allez dans Network, Wireless puis SSIDs. Ici :

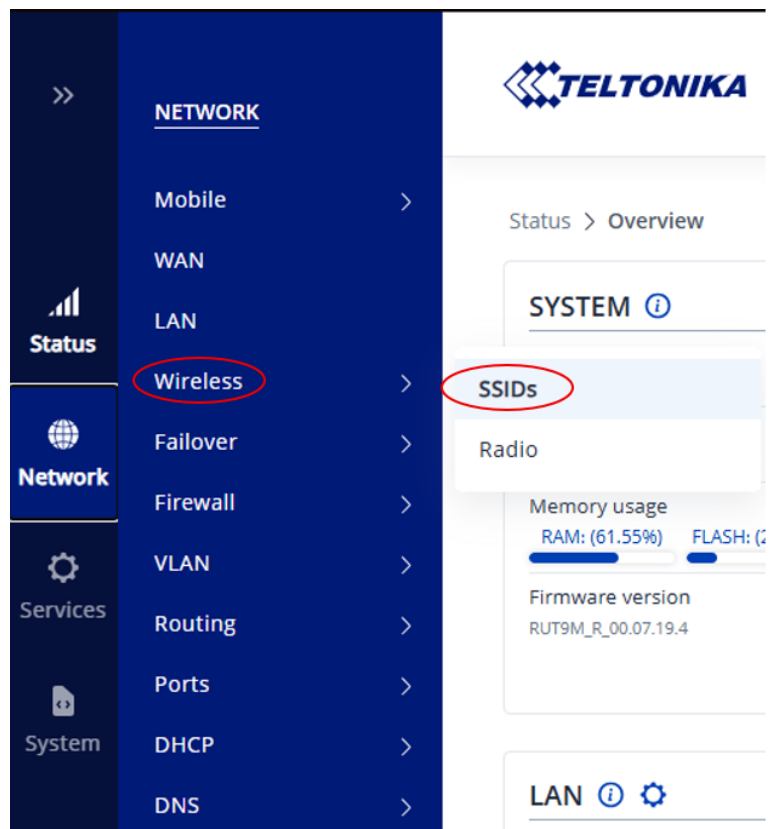


FIGURE 5 – Chemin d'accès SSID

Puis cliquez sur **Scan 2.4GHz**, sélectionnez la connexion que vous voulez utiliser (Unice Hotspot). Chaque robot ne peut-être connecté qu'à **une seule autre connexion**. Pour utiliser Unice Hotspot, connectez vous dessus. Puis allé ici :

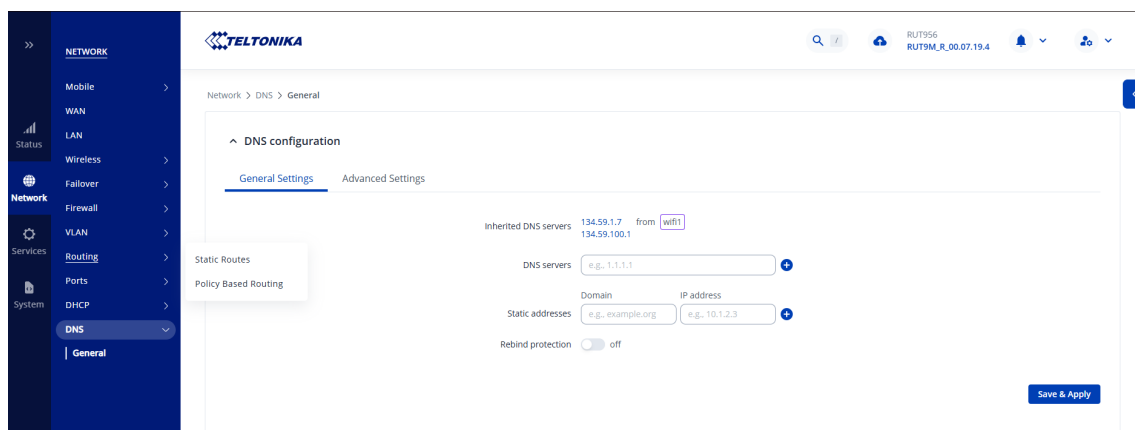


FIGURE 6 – Option DNS

Une fois connectez, essayez de vous connecter à n'importe quel site internet, on vous redirigera vers la page Unice usuelle.

Connection SSH :

1. Via noVNC : <https://192.168.1.102/>, le mot de passe est **ROSKit** pour tout les robots.. Vous êtes connecté sur l'ordinateur de bord du robot. Sur ce site vous aurez tous les **retours visuels des manipulations** de ce TP. Ouvrez un terminal (voir figure 6), vous pouvez taper « code . ». Une fenêtre VSCode devrait s'ouvrir.

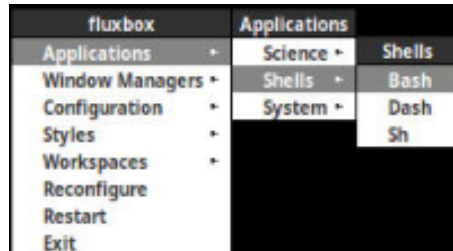
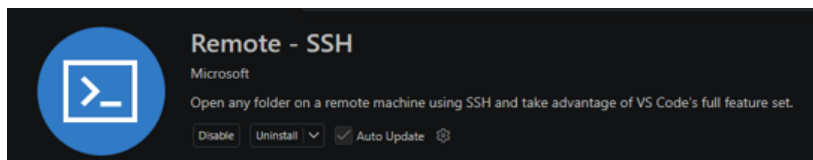
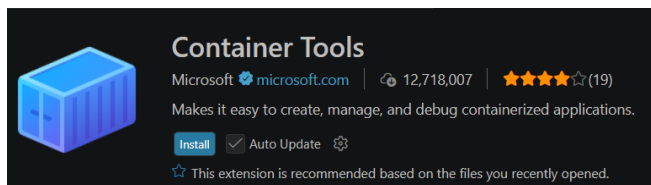


FIGURE 7 – Ouvrir un shell

2. Via VSCode Remote-SSH : Téléchargez Remote - SSH et Container Tool (voir figure 7) sur VSCode



(a) Extension Remote - SSH



(b) Extension Container Tool



(c) Option VSCode

FIGURE 8 – Extensions Information

Dans les options de VSCode on retrouve **3 sections** intéressantes (voir figure 7.c). La première pour se déplacer dans nos **fichiers** (en haut), la deuxième est le **remote explorer** (au milieu) et la troisième l'**accès au docker** (en bas). Pour l'instant, rendez-vous sur le remote explorer et appuyez sur le "+" pour **ajouter un connection SSH**. Dans la barre de recherche en haut tapez : `user@nom_du_robot.local`

Exemples :

- (a) `user@grkit-007.local` - pour le robot 7
- (b) `user@grkit-011.local` - pour le robot 11

Le mot de passe est **ROSKitGR** pour tous les robots. Pour finir, sélectionnez le **répertoire** dans lequel vous voulez travailler. Ici il s'agit de `/home/user/ros-kit_ws/src`. Une fois dedans vous verrez tous les packages ros2 déjà installés.

Initialisation Docker : Accédez au conteneur :

```
1 docker exec -it gr-p247-humble-container bash
2 # Verifiez que les topics sont publies :
3 ros2 topic list
```

2 Navigation Autonome avec Nav2

Cette partie s'effectue exclusivement dans les dockers. N'oubliez pas de sourcer votre environnement si ce n'est pas déjà fait : `source /opt/ros/humble/setup.bash`.

2.1 Packages nécessaires

```
1 # Installation des outils
2 sudo apt update
3 sudo apt install -y \
4     ros-humble-navigation2 \
5     ros-humble-nav2-bringup \
6     ros-humble-pointcloud-to-laserscan \
7     ros-humble-slam-toolbox \
8     python3-colcon-common-extensions \
9     python3-rosdep
10
11 cd ~/roskit_ws/src
12
13 # Clonage de Navigation2 (Branche Humble)
14 git clone https://github.com/ros-navigation/navigation2.git -b
    humble
15
16 cd ~/roskit_ws
17 sudo rosdep init
18 rosdep update
19 rosdep install --from-paths src --ignore-src -r -y
20
21 # On desactive les flags qui pourraient forcer le multi-cœur
22 unset MAKEFLAGS
23
24 # Compilation globale limitée à 1 worker, les paquets étant très
    massif, il pourrait faire crash le robot.
25 colcon build --symlink-install \
26     --cmake-args -DCMAKE_BUILD_TYPE=Release \
27     --parallel-workers 1
28
29 # On lance tous les programmes
30 source install/setup.bash
31 ros2 launch gr_bringup main.launch.py
```

Vérifiez que le robot publie. Dans noVNC, regarder si la caméra renvoie bien des images et le lidar aussi. Si tout est statique vous allez certainement devoir relancer le robot. Vous n'avez pas à tout retélécharger.

Vous pouvez vérifier si les paquets sont bien installés dans la partie docker (voir figure 7.c).

2.2 Conversion PointCloud vers LaserScan

Le Lidar du robot publie un nuage de points 3D. Nav2 nécessite du 2D. On veut donc transformer les points 3D. En effet, il publie dans `/rslidar_points` un nuage de points

(visible dans rviz). Le problème est qu'il publie des points au sol. Ces points pourront être considéré comme des murs au moment de la construction de la carte. Ainsi, pour construire notre carte 2D on ne va considérer que les points compris entre -0.1m et 0.5m par rapport au Lidar.

```
1 source /opt/ros/humble/setup.bash
2 ros2 run pointcloud_to_laserscan pointcloud_to_laserscan_node \
3   --ros-args \
4   -r cloud_in:=/rslidar_points \
5   -p target_frame:=base_link \
6   -p min_height:=-0.1 \
7   -p max_height:=0.5 \
8   -p range_min:=0.1 \
9   -p range_max:=30.0
```

Vérifiez la publication des données sur `/scan` avec `ros2 topic echo /scan`.

2.3 Cartographie (SLAM)

Afin de naviguer, nav2 à besoin d'une carte. Pour l'instant nous n'en avons pas. Nous allons utiliser `slam_toolbox` pour en faire une à partir de l'odométrie du robot et du laserscan récupéré juste au-dessus. L'odométrie permet d'estimer la position du robot très rapidement mais reste imprécise, la node la corrige avec les valeurs du laserscan. On utilise le mode **online asynchrone** afin de créer notre carte de façon dynamique. Le robot peut bouger tout en créant la carte. Si le processeur est très sollicité, le programme préférera ignorer quelques points laser plutôt que de retarder l'estimation de la position.

Utilisez `slam_toolbox` pour générer la carte :

```
1 ros2 launch slam_toolbox online_async_launch.py
```

Il se peut que vous rencontriez des problèmes liés à l'odométrie. Veuillez vérifier que la `base_frame` est bien `base_link` (*voir figure 8*). En effet, l'odométrie est transformée en `base_link`, mais il se peut que la `base_frame` soit initialisée à un autre nom.

Dans le docker, faites coulisser et cherchez :

\opt\ros\humble\share\slam_toolbox\config\mapper_params_online_async.yaml

```
1  slam_toolbox:
2    ros_parameters:
3
4    # Plugin params
5    solver_plugin: solver_plugins::CeresSolver
6    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
7    ceres_preconditioner: SCHUR_JACOBI
8    ceres_trust_strategy: LEVENBERG_MARQUARDT
9    ceres_dogleg_type: TRADITIONAL_DOGLEG
10   ceres_loss_function: None
11
12   # ROS Parameters
13   odom_frame: odom
14   map_frame: map
15   base_frame: base_link
16   scan_topic: /scan
17   use_map_saver: true
18   mode: mapping #localization
```

FIGURE 9 – Fichier yaml de online_async

2.4 Lancement de la navigation

Assurez-vous que votre fichier : \opt\ros\humble\share\nav2_bringup\params\nav2_params.yaml est correct. Vous pouvez en récupérer un valide ici : [Lien GitHub](#).

Lancez la navigation :

```
1 ros2 launch nav2_bringup navigation_launch.py
```

Dans RViz, utilisez l'outil "Nav2 Goal" pour définir une destination.

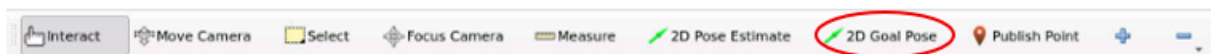


FIGURE 10 – Paramètres rviz

2.5 Enregistrement de la carte

Pour visualiser la carte que vous êtes en train de créer allez dans rviz et faite : **ADD -> By Topic -> /map -> Map**.

Cela permet de visualiser le retour du **/map** et de se repérer. Ce qui vérifie que le programme fonctionne bien.

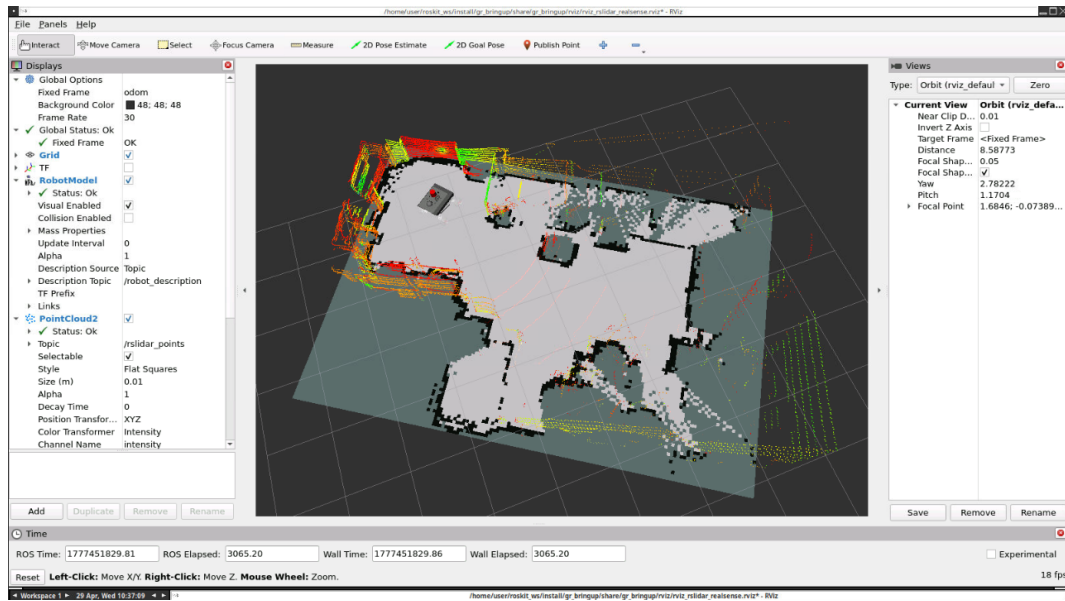


FIGURE 11 – Rendu de la carte au sol

Maintenant, pour voir les trajectoires du robot et faire de la navigation autonome nous allons utiliser la default view de nav2 :

```
1 ros2 run rviz2 rviz2 -d /opt/ros/humble/share/nav2_bring_up/rviz/
  nav2_default_view.rviz
```

Vous devriez avoir un rendu avec beaucoup de d'information :

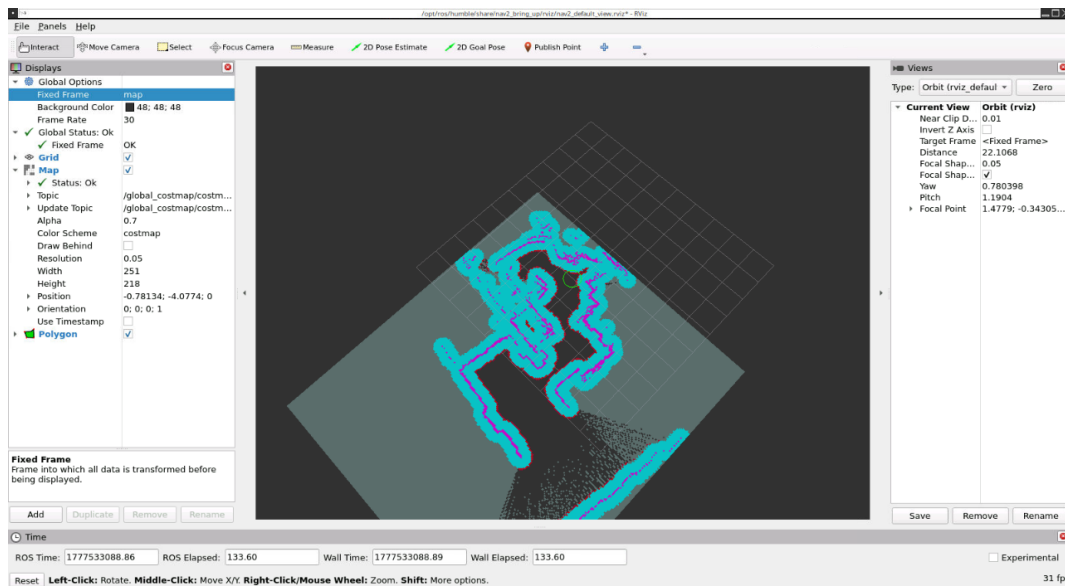


FIGURE 12 – Rendu de la carte au sol

Vous pouvez décocher certaines options pour mieux y voir. Ces options permettent de savoir ce que prend en compte le robot pour effectuer le pathfinding. Si rien ne s'affiche aller dans les topics et ajouter la "map" et le "polygone" du **global costmap**, le "path" et goal si vous le voulez. Faites dérouler la map dans le menu et choisissez "costmap", vous verrez comme sur la figure 11, les bordures. Cela permet de voir clairement si le

robot est bloqué ou non. En violet sont les murs, en bleu les murs avec l'inflation, et en rouge le bord de l'inflation.

Si robot submergé : Dans le fichier `nav2_params.yaml`, allez a `global costmap` et réduisez **`inflation_radius`** à 0.55 et le **`robot_radius`** a 0.35. Maintenant relancer votre `nav2_bringup`, activez la carte, l'inflation des mur et la taille du robot d'avoir permettre plus de mouvements.

Au bout de quelques rotations, le robot se perd. En effet, l'odométrie est imprécise. Pour corriger ces valeurs voir le le TP sur l'IMU.

Pour sauvegarder votre travail : Vous pouvez effectuer :

```
1 ros2 run nav2_map_server map_saver_cli -f nom_de_la_carte
```